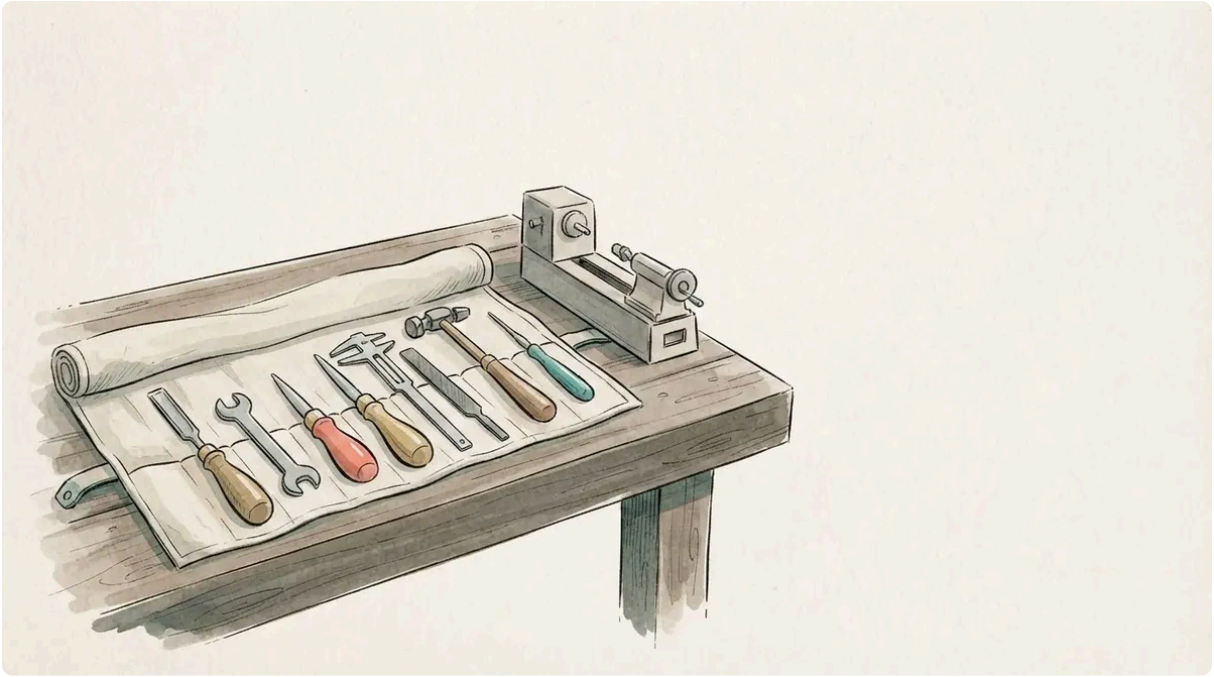
Zola has no plugins, so this blog grew a Rust CLI

Emil Lindfors | January 27, 2026



Here is what a deploy of this blog looks like from the terminal, with the per-post lines cut out:

```
$ ./build.sh
Processing citations...
Generating markdown representations...
Generating speech scripts...
Generating audio...
Generating CV...
Generating PDFs...
Building site with Zola...
  (no local zola -- using ghcr.io/getzola/zola:v0.23.2)
Done!
```

One script, and everything the site does comes out of it. Each post gets a typeset PDF, a plain-markdown copy for anyone who asks for one with an `Accept` header, a spoken script, and for some posts an MP3. Citations resolve to a formatted reference list with DOIs. The CV compiles. Then Zola renders the lot, and a push puts it on Cloudflare Pages.

There is a newsletter behind it too, on a Rust Worker and a mail server I run myself, and a client-side search that never calls anyone. No JavaScript framework, no `node_modules`, no third-party service except Cloudflare.

This post is the foundation of the series: why Zola, what Zola refuses to do, and how that one refusal decided the shape of everything around it. The newsletter, the citations, the PDFs, the images and the audio each have their own post.

Why Zola

I looked at Hugo, Astro, Next.js and Zola in January.

I wanted to use Zola because I enjoy projects built in Rust, and I feel quite confident with the dependencies of this project and that it is battle tested. I was thinking of using more advanced projects like Dioxus, but that would have been overkill for a place to write about stuff.

Beyond that, four things decided it.

One binary. Zola is a single executable. No Node, no Ruby, no Python, no lockfile. Hugo has the same property. Astro and Next.js bring the whole Node ecosystem with them, and with it version managers, `npm audit`, and a `node_modules` directory heavier than the blog.

Batteries included. Sass compilation, syntax highlighting through syntect, an Atom feed, and a search index. The search index is a JavaScript file that assigns one global variable, `window.searchIndex`, holding a prebuilt elasticlunr index of every post. `search.js` loads it and queries it in the browser. That technique is older than most JavaScript frameworks and it is exactly the right amount for a blog with twelve posts. No Algolia, no server, no API key.

Tera templates. Tera is Jinja2 for Rust. If you have written a Django or Ansible template, you already know it. It has template inheritance, components with typed arguments, and list comprehensions. The reference list, the newsletter form and the series navigation are all components in one file.

TOML frontmatter. Posts open with `+++` and TOML, not `---` and YAML. That matters here because a post's resolved citations are an array of structured records in its own frontmatter, and TOML's array-of-tables carries that without indentation games.

Also, it is fast. The whole site renders in well under a second, and `zola serve` reloads on save. I never wait for it.

What Zola won't do

Zola is deliberately small. It has:

- **No asset processing.** No image optimisation, no CSS minification, no bundling.

- **No client-side routing.** Every page is a full HTML document. That is fine.
- **No plugin system.** Unlike Hugo or Gatsby, there is no way to extend it from the inside.
- **No newsletter, no email, no CMS.** Content is files.

The third one is the decision that shaped everything else. Since Zola cannot be extended internally, anything it does not do has to happen *before* it runs, on the files it is about to read. Citations are resolved into the post's frontmatter. PDFs are generated into `static/`. The markdown copies, the speech scripts and the audio go into `static/` too. Zola then sees clean markdown and structured data, and knows nothing about any of it.

In February that pre-processing was a handful of shell scripts. Since August it is one Rust CLI.

The tool that grew instead

`site-tools` lives in `tools/site-tools/`, about 6,500 lines of Rust in fifteen files, and it owns every generated file in the repo:

Subcommand	Writes	Needs
`cite all`	resolved references into each post's frontmatter	crossref, or a local Zotero library, once
`markdown all`	`static/blog/<slug>.md`, `static/llms.txt`, `static/newsletter/recent.json`	nothing
`speech all`	`static/speech/<slug>.txt`	nothing
`audio all`	`static/audio/<slug>.mp3`	ffmpeg and a TTS endpoint
`cv build`	`static/cv.pdf`	typst and the font sources
`pdf all`	`static/pdf/<slug>.pdf`	typst and the font sources
`newsletter gen` / `send`	`static/newsletter/<slug>.md`, then a POST to the newsletter service on the mail server	the admin key

Three modules are shared by all of them, and they are why each new subcommand was cheaper than the one before:

- `frontmatter.rs` splits a post on the `+++` delimiters and parses the TOML, so `draft` is a real boolean everywhere. A `grep '^draft'` matches `draft = "true"` just as happily, and drafts leaking into `static/pdf/` is how the shell version got replaced.
- `bib.rs` formats a reference record and strips the citation anchors. The PDF, the markdown copy and the speech script all call it.
- `codemask.rs` hides every fenced block and inline span behind a NUL-wrapped placeholder before any text pass runs, so a post that documents the `@citekey` syntax does not get its examples rewritten into citations.

One rule holds it together: **everything generated is committed.** `git status` after a build shows the PDF, the markdown copy and the speech script next to the post. Two reasons:

1. Cloudflare Pages builds from git and runs no install step, so a file it cannot find in the repo does not exist.
2. The committed post is the citation cache. After the first `cite` run there are no `@key` markers left, so a rebuild touches neither the network nor Zotero, and a fresh clone builds without either.

The cost is a fatter repo. The PDFs and the MP3s are binary files in git history for good, and I made that choice knowing it. I ran the audio pipeline over the whole back catalogue once to see what it came to, and got 97 minutes and 45 MB. It is still switched off.

I think I'm still undecided on the audio part, if it's really worth it, but it doesn't seem too bad: I don't pay anything for the site and it builds really quickly.

The upgrade that cost something

Zola 0.23 arrived in August, and it sent two bills.

Tera v2. Macros were removed in favour of components, and the `concat` and `filter` filters went with them. The related-posts block was built on those two filters. It is now a list comprehension, and shorter, but every template needed a pass. I ran `pdf all` after the migration and got zero diff against the committed PDFs, which is how I knew the content side had survived. While doing the template pass I found that CI was still pinned to Zola 0.19.2, which cannot even find a `zola.toml`, so CI had not built the site in months. Nothing had said so.

Windows. Zola 0.23 canonicalises the project root to a `\\?\` UNC path and then globs for templates under it, and the glob matches nothing (#3229). So on my Windows machine there is no local `zola` at all. `scripts/lib.sh` checks for a 0.23.x binary and otherwise runs the pinned docker image, which is what the `(no local zola ...)` line in the opening transcript is about. Live reload does not survive a Windows bind mount either, so `zola serve` there means re-running after each edit. It works, and it is the one part of the setup I would not have planned.

Neither was hard. Both were paid for in code that exists because Zola has no plugins, and that is the pattern the next section is about.

A path, in three phases

There is a name for what happened to this repo. Sydow, Schreyögg and Koch (2009) took path dependence, the idea that where you end up is decided by where you started, and opened it into a process with three phases. At the start the options are wide open, and a small choice narrows them. They call that the critical juncture. Then self-reinforcing

mechanisms take over: each step makes the next step in the same direction cheaper, through learning, through pieces that fit together, and through everyone's expectations lining up. At the end you are locked in. The path holds even when a better option is in plain view, because leaving means paying for everything that was built on it. The textbook example is the QWERTY keyboard, which nobody would design today and nobody will replace.

They wrote about firms, and this is one person and a hobby repo, so treat the transfer as an analogy that fits well and not as a finding. But it does fit well.

- **The critical juncture** was small: Zola has no plugins, so citations get resolved into the frontmatter before the build. That was one shell script in February.
- **The self-reinforcement** was the shared code. Once `frontmatter.rs` and `bib.rs` existed, the PDF generator, the markdown exporter and the speech script could each reuse the split, the parse and the reference formatting, and none of them had to solve those again. Each one also reinforced the rule that generated files are committed, because each one produced files that Pages could not otherwise find. By the fourth subcommand there was no decision left to make about where a new feature would go.
- **The lock-in** is real and I can name it. The site depends on Zola's page bundles, its TOML frontmatter, Tera v2, Cloudflare's `_headers` and Pages Functions, and Stalwart's JMAP. The Tera migration was the first bill for that.

I thought in January that I would be more dependent on Cloudflare infrastructure, but I keep building away from that, so I could probably run this site on my own server without much hassle in a while if I really wanted to. (The newsletter did exactly that in September, in a day.) Cloudflare works fine now, but I do enjoy not being locked down, as I have written about in other posts as well, on the need to be able to experiment and innovate and not to be locked in, as they say in the path dependency literature.

The model makes two recommendations that I find useful here, and one point where I would push back.

1. **Notice the small choices.** The decisions that lock you in do not look like decisions. Putting generated files in `static/` was one. Making the committed post the cache was another. Write them down when you make them.
2. **Judge lock-in by the exit, not by how popular the tool is.** The question is what leaving would cost. Here the content is markdown and TOML in git, and the plain-markdown copies in `static/blog/` are already a complete export. The templates and the CLI are the sunk part. Moving the posts to another generator would be a small job. Rewriting the tooling would not. I can live with that.
3. **The path is also what makes it maintainable.** The theory treats lock-in as a loss of flexibility, full stop. On a project this size, every feature going into the same binary, with the same tests, run by the same script, is what lets one person run a site with a newsletter, PDFs and audio and still remember how it works. The lock-in to watch is

the other kind, to somebody else's platform, and that is the one I keep building away from.

The repo

```
lindfors-site/
├── content/blog/<slug>/index.md   # post: TOML frontmatter + markdown, images
alongside
├── templates/                     # base, page, index, search, components,
series/
│   └── pdf/academic.typ           # Typst template for the post PDFs
├── sass/style.scss                # every style, both themes, ~2,100 lines
├── static/
│   ├── fonts/                     # Inter (variable) and Literata (subset),
woff2
│   ├── js/                        # theme, post chrome, search, newsletter, rum
│   ├── blog/<slug>.md             # generated: plain markdown, content
negotiation
│   ├── pdf/, speech/, audio/      # generated
│   ├── newsletter/                # generated: issues and recent.json
│   ├── llms.txt                   # generated
│   └── _headers                   # security headers and the CSP
├── functions/blog/_middleware.js  # Pages Function: Accept: text/markdown →
the .md
├── newsletter/                    # axum service on the mail server: subscribe,
send, dashboard
├── tools/site-tools/              # the Rust CLI
├── tools/img-optim/               # image conversion to WebP, run by hand
├── scripts/lib.sh                 # run_zola, the preflights, the docker
fallback
├── cv.typ
├── build.sh                       # the one definition of a build
├── deploy.sh                      # build.sh, then commit and push
└── zola.toml
```

The blog, the templates, the styles, the newsletter service, the CLI, the build script and the CV source are one repo. Everything except the newsletter service deploys from a push; that one is a binary copied to the mail server.

Typography

Two typefaces, both self-hosted from `/fonts/`. No Google Fonts CDN, no third-party request.

- **Literata** for body text. A serif designed for long-form reading on screens, originally for Google Play Books, with a generous x-height. It is a great face for reading 2,500 words in one sitting. Four weights and the italic, subset to Latin with `pyftsubset` and served as woff2.
- **Inter** for headings, navigation and UI. One variable woff2 and its italic.

The subsetting command, if you want it:

```
uvx --from fonttools --with brotli pyftsubset \
  Literata-Regular.ttf \
  --output-file=Literata-Regular.woff2 \
  --flavor=woff2 \
  --layout-features='*' \
  --
unicodes='U+0000-00FF,U+0131,U+0152-0153,U+02BB-02BC,U+02C6,U+02DA,U+02DC,U+2000-206F,U+2074,U+
```

The `@font-face` declarations are inlined in a `<style>` block in the `<head>`, so the browser finds them before it fetches the stylesheet and the text does not reflow when the fonts arrive. Typst needs the TTF sources for the PDFs. Those are gitignored and fetched by `scripts/fetch-fonts.sh`, so the same two faces appear on the page and on paper.

Two themes

Light is *Sandy Shore*, warm beige with deep navy text. Dark is *Deep Ocean*, blue-green with light text. Both use coral and teal as accents. The whole thing is CSS custom properties keyed on a `data-theme` attribute:

```
:root {
    --color-bg: #F0EAE0;               // Sandy Shore
    --color-text: #1C3240;             // Deep Sea
    --color-link: #D4706A;             // Coral
    --color-accent-secondary: #2A8F82; // Teal
}

[data-theme="dark"] {
    --color-bg: #0E1A20;               // Deep Ocean
    --color-text: #E8F0F0;
    --color-link: #F2A07B;             // Warm Coral
    --color-accent-secondary: #4DD4AC; // Bright Teal
}
```

Every colour in the stylesheet goes through these variables. The toggle is one `setAttribute` and one `localStorage.setItem`. On load the script reads `localStorage` first and falls back to `prefers-color-scheme`.

One thing to note here. That script used to be inline in the body, so it ran before the first paint. The site now ships a Content-Security-Policy with `script-src 'self'` and no `unsafe-inline`, because Pages serves headers statically and there is no per-request nonce to be had. So every inline script moved into `static/js/`, and `theme.js` loads at the end of the body. In theory a dark-mode reader with a saved preference can see one frame of beige on a slow connection. I have not managed to see it.

The post page

Two columns: the post on the left, a sticky sidebar on the right. The sidebar holds:

- **Table of contents** with scroll-spy, so the current section is highlighted as you read.
- **Date, reading time, and the changelog.** A revised post gets a dated entry, and the latest one shows here. I do not silently rewrite published posts.
- **PDF, Cite and Share.** Cite opens a modal with BibTeX and APA for *the post*, with the PDF as the URL and today's date as the access date, so a post can go into a reference list.
- **Tags** and an author card with the CV link.

Above the post, when the build produced one, is an audio player with playback speeds. And there is a third representation you cannot see. Ask for a post with `Accept: text/markdown` and a Pages Function serves the plain-markdown copy from `static/blog/` at the same URL. It is also addressable directly as `/blog/<slug>.md`, a `Link` header advertises it, and `llms.txt` lists the lot. A link an agent was given still resolves for a human, and a human's link still works for an agent.

The scroll-spy and the reading-progress bar share one passive scroll listener. The site's own JavaScript is six small files, about 16 KB in total. The monitoring SDK is a seventh, fetched only after the page has loaded and only for sampled visits.

The build, in order

```
site-tools cite all        # @key markers → [[extra.references]] in the post
site-tools markdown all    # static/blog/<slug>.md, llms.txt, recent.json
site-tools speech all      # static/speech/<slug>.txt
site-tools audio all       # static/audio/<slug>.mp3, only when a script changed
site-tools cv build        # static/cv.pdf
site-tools pdf all         # static/pdf/<slug>.pdf, drafts skipped
run_zola build             # local zola 0.23.x, or the docker image
```

The order is not optional, and you will want to keep it if you copy this. Citations first, because the markdown copy, the speech script and the PDF all render the resolved references and must match the HTML. Audio after speech, and gated on the script's hash, so

an unchanged post costs nothing and needs no endpoint. PDFs before Zola, because they land in `static/` and Zola copies `static/` to `public/`.

Two preflights run before any of it:

- **`typst` on PATH and font sources in `fonts/`.** Without them every regenerated PDF comes out in fallback fonts. The old script warned once per post and carried on, and the site looked freshly built with stale PDFs on it.
- **No unconverted JPEG under `content/`.** `deploy.sh` runs `git add -A`, and a 4 MB photo is in the history for good the moment that happens.

`SKIP_PDFS=1` and `SKIP_AUDIO=1` turn the expensive halves off, and you will use both while editing. With both off, a build is a few seconds. With audio on and a script changed, it is however long the TTS endpoint takes.

The cost

Component	Monthly
Cloudflare Pages	$0
Mail server, its share of a VPS I already run	~$6-7
Domain	~$0.83

Under $8 a month for a blog with a newsletter, PDFs, audio, client-side search and self-hosted fonts. The VPS also runs the identity provider, the log store and a few other things, so the mail server's marginal cost is close to zero.

The $5 VPS has increased slightly, to 6 or 7 dollars, but it's still fine for a box that does mail, auth, vault etc.

What I'd do differently

Start with the typography. I picked colours first and fonts second. Literata's warmth is what pushed the palette to beige, and I got there on the fourth attempt. The font constrains everything else, so choose it first.

Ship with system preference only. `prefers-color-scheme` is enough for launch. I spent a day on the toggle, the persistence and the flash-avoidance, and the CSP later undid part of it. Add the toggle when someone asks for it.

Use page bundles from the start. `blog/my-post/index.md`, not `blog/my-post.md`. The directory form lets images sit next to the post, and I had to migrate to it the day the first photo arrived.

One build script. `build.sh` and `deploy.sh` used to carry their own copies of the citation and PDF loops, and they drifted. `deploy.sh` once compiled the CV without `--font-path`, so

Typst fell back to whatever fonts the machine had. Now `deploy.sh` is `build.sh` followed by a commit and a push, and there is nothing left to disagree.

The stack

Role	Tool
Static site generator	Zola 0.23
Hosting	Cloudflare Pages, plus one Pages Function for content negotiation
Newsletter	one axum service on the mail server, Postgres behind it (the move)
Mail server	Stalwart, over JMAP on loopback
Everything generated	`site-tools` (Rust)
PDFs	Typst, via cmarker and MiTeX
Audio	Fish Audio, through `site-tools audio`
Real user monitoring	OpenObserve, self-hosted
Body font	Literata
Heading font	Inter
Search	elasticlunr, on the index Zola builds
Math	KaTeX on the web, MiTeX in the PDF
Styles	Sass, compiled by Zola

Everything is text in a git repo. No CMS, no database, no admin panel on the public site. I write markdown in an editor, run one script, and push. The source is on GitHub.

References

- Sydow, J., Schreyögg, G., & Koch, J. “Organizational Path Dependence: Opening the Black Box”. *Academy of Management Review*, vol. 34, no. 4, pp. 689-709, 2009. doi:10.5465/amr.2009.44885978